JOBSHEET X

Mata Kuliah: Praktikum Algoritma Dan Struktur Data

Queue

Dosen Pengampu:

Mungki Astiningrum, S.T., M.Kom.

Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 1G

D-IV Teknik Informatika

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2025-2026

2. Praktikum 1

2.1 Percobaan 1: Operasi Dasar Queue

- **Queue03**

```
package Jobsheet10;

public class Queue03 {
    int[] data;
    int front;
    int rear;
    int size;
    int max;

    public Queue03(int n) {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }

    public boolean IsFull() {
        if (size == max) {
            return true;
        } else {
            return false;
        }
    }

    public void peek() {
        if (!IsEmpty()) {
            System.out.println("Elemen terdepan: " + data[front]);
        } else {
            System.out.println("Queue masih kosong");
        }
    }

    public void print() {
```

```
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.print(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}

public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}

public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
```

```
        } else {
            dt = data[front];
            size--;
            if (IsEmpty()) {
                front = rear = -1;
            } else {
                if (front == max - 1) {
                    front = 0;
                } else {
                    front++;
                }
            }
        }
        return dt;
    }
}
```

- **QueueMain03**

```
package Jobsheet10;

import java.util.Scanner;

public class QueueMain03 {

    public static void menu() {
        System.out.println("Masukkan operasi yang diinginkan:");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Print");
        System.out.println("4. Peek");
        System.out.println("5. Clear");
        System.out.println("-----------------");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan kapasitas queue: ");
        int n = sc.nextInt();

        Queue03 Q = new Queue03(n);
        int pilih;

        do {
            menu();
            pilih = sc.nextInt();
```

```
            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
                    int dataMasuk = sc.nextInt();
                    Q.Enqueue(dataMasuk);
                    break;
                case 2:
                    int dataKeluar = Q.Dequeue();
                    if (dataKeluar != 0) {
                        System.out.println("Data yang dikeluarkan: " + dataKeluar);
                    }
                    break;
                case 3:
                    Q.print();
                    break;
                case 4:
                    Q.peek();
                    break;
                case 5:
                    Q.clear();
                    break;
            }
        } while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih == 5);

        sc.close();
    }
}
```

2.1.2. Verifikasi Hasil Percobaan

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>  & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInEx
ceptionMessages' '-cp' 'C:\Users\IdeaPad\AppData\Roaming\Code\User\workspaceStorage\125289802138902 5de57e0e2ce7124bb\redhat.java\jdt_ws\Praktik Algo&StrukturData_7951cb7c\bin
' 'Jobsheet10.QueueMain03'
Masukkan kapasitas queue: 4
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
----------------
1
Masukkan data baru: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
----------------
1
Masukkan data baru: 31
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
----------------
4
Elemen terdepan: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
----------------
```

2.1.3. Pertanyaan

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?
2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {
    rear = 0;
```

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {
    front = 0;
```

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?
5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

6. Tunjukkan potongan kode program yang merupakan queue overflow!
7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawab

1. Pada saat konstruktor dipanggil, nilai size diatur menjadi 0 karena memang belum ada data yang dimasukkan ke dalam antrean (antrean kosong). Sementara itu, nilai front (depan) dan rear (belakang) diatur menjadi -1 sebagai penanda absolut bahwa antrean benar-benar dalam keadaan kosong. Karena indeks *array* dimulai dari 0, angka -1 digunakan untuk menunjukkan bahwa penunjuk tersebut belum menunjuk ke indeks data mana pun.
2. Potongan kode ini merupakan antrean. Jika posisi rear (penunjuk elemen paling belakang) sudah mencapai batas akhir indeks *array* (max - 1), sedangkan antrean sebenarnya belum penuh karena ada data di bagian depan yang sudah dikeluarkan, maka posisi rear akan dikembalikan ke indeks 0. Hal ini bertujuan untuk mengisi ruang kosong di awal *array* agar memori dapat dimanfaatkan kembali.
3. Mirip dengan potongan kode di no. 2. Ketika sebuah data yang berada di posisi akhir indeks array (max - 1) dikeluarkan (melalui operasi Dequeue), penunjuk front (elemen terdepan) tidak dapat bergeser lagi ke sisi kanan.

Oleh karena itu, posisi front dikembalikan ke indeks 0 untuk menunjuk antrean terdepan berikutnya yang berada di awal array.

4. Pada struktur data Queue, elemen pertama tidak selalu berada di indeks 0. Seiring berjalannya proses penambahan dan pengeluaran data, posisi elemen terdepan akan terus bergeser secara dinamis. Oleh karena itu, perulangan harus dimulai dari posisi front untuk memastikan program mencetak elemen sesuai dengan urutan antrean yang sebenarnya, bukan sekadar mencetak isi memori dari awal array.
5. Potongan kode ini berfungsi sebagai penunjuk (*iterator*) yang bergerak secara melingkar. Operasi *modulo* (% max) berfungsi untuk memastikan bahwa apabila nilai i telah mencapai batas maksimal ukuran *array*, nilainya akan diatur ulang (*reset*) menjadi 0. Dengan demikian, proses perulangan dapat terus membaca sisa elemen di awal *array* tanpa mengalami galat batas indeks (*Index Out of Bounds*).
6. potongan kode program yang merupakan queue overflow (antrean penuh)

```
if (IsFull()) {
    System.out.println("Queue sudah penuh");
}
```

7. Modifikasi

```
65    public void Enqueue(int dt) {
66        if (IsFull()) {
67            System.out.println(x: "Queue sudah penuh");
68            System.exit(status: 0);
69        } else {
70            if (IsEmpty()) {
71                front = rear = 0;
72            } else {
73                if (rear == max - 1) {
74                    rear = 0;
75                } else {
76                    rear++;
77                }
78            }
79            data[rear] = dt;
80            size++;
81        }
82    }
83
84    public int Dequeue() {
85        int dt = 0;
86        if (IsEmpty()) {
87            System.out.println(x: "Queue masih kosong");
88            System.exit(status: 0);
89        } else {
90            dt = data[front];
91            size--;
92            if (IsEmpty()) {
93                front = rear = -1;
94            } else {
95                if (front == max - 1) {
96                    front = 0;
97                } else {
98                    front++;
99                }
100           }
101       }
102       return dt;
```

2.2. Percobaan 2: Antrian Layanan Akademik

- **AntrianLayanan03**

```
package Jobsheet10.Praktikum2;

public class AntrianLayanan03 {
    Mahasiswa03[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan03(int max) {
        this.max = max;
        data = new Mahasiswa03[max];
        size = 0;
        front = 0;
        rear = -1;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }

    public boolean IsFull() {
        if (size == max) {
            return true;
        } else {
            return false;
        }
    }

    public void peek() {
        if (!IsEmpty()) {
            System.out.println("Elemen terdepan: " + data[front]);
        } else {
            System.out.println("Queue masih kosong");
        }
    }

    public void print() {
        if (IsEmpty()) {
```

```
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.print(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}

public void clear() {
    if (!IsEmpty()) {
        front = 0;
        rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void tambahAntrian(Mahasiswa03 mhs) {
    if (IsFull()) {
        System.out.println("Antrian penuh, tidak dapat menambah mahasiswa");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = mhs;
    size++;
    System.out.println(mhs.nama + " berhasil ditambahkan ke antrian");
}

public Mahasiswa03 layaniMahasiswa() {
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
        return null;
    }
    Mahasiswa03 mhs = data[front];
    front = (front + 1) % max;
    size--;
    return mhs;
}

public void lihatTerdepan() {
```

```
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.print("Mahasiswa terdepan: ");
            System.out.println("NIM - NAMA - PRODI - KELAS");
            data[front].tampilkanData();
        }
    }

    public void tampilkanSemua() {
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Daftar Mahasiswa dalam Antrian:");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        for (int i = 0; i < size; i++) {
            int index = (front + i) % max;
            System.out.print((i + 1) + ". ");
            data[index].tampilkanData();
        }
    }

    public int getJumlahAntrian() {
        return size;
    }
}
```

- **LayananAkademikSIAKAD03**

```
package Jobsheet10.Praktikum2;

import java.util.Scanner;

public class LayananAkademikSIAKAD03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan03 antrian = new AntrianLayanan03(5);
        int pilihan;

        do {
            System.out.println("\n=== Menu Antrian Layanan Akademik ===");
            System.out.println("1. Tambah Mahasiswa ke Antrian");
            System.out.println("2. Layani Mahasiswa");
            System.out.println("3. Lihat Mahasiswa Terdepan");
            System.out.println("4. Lihat Semua Antrian");
            System.out.println("5. Jumlah Mahasiswa dalam Antrian");
            System.out.println("0. Keluar");
```

```
            System.out.print("Pilih menu: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("NIM   : ");
                    String nim = sc.nextLine();
                    System.out.print("Nama  : ");
                    String nama = sc.nextLine();
                    System.out.print("Prodi : ");
                    String prodi = sc.nextLine();
                    System.out.print("Kelas : ");
                    String kelas = sc.nextLine();

                    Mahasiswa03 mhs = new Mahasiswa03(nim, nama, prodi, kelas);
                    antrian.tambahAntrian(mhs);
                    break;
                case 2:
                    Mahasiswa03 dilayani = antrian.layaniMahasiswa();
                    if (dilayani != null) {
                        System.out.print("Melayani mahasiswa: ");
                        dilayani.tampilkanData();
                    }
                    break;
                case 3:
                    antrian.lihatTerdepan();
                    break;
                case 4:
                    antrian.tampilkanSemua();
                    break;
                case 5:
                    System.out.println("Jumlah dalam antrian: " +
antrian.getJumlahAntrian());
                    break;
                case 0:
                    System.out.println("Terima kasih.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
        } while (pilihan != 0);

        sc.close();
    }
}
```

- **Mahasiswa03**

```java
package Jobsheet10.Praktikum2;

public class Mahasiswa03 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa03(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }
}
```

2.2.2 Verifikasi Hasil Percobaan

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM   : 123
Nama  : aldi
Prodi : ti
Kelas : 1a
aldi berhasil ditambahkan ke antrian

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM   : 124
Nama  : bobi
Prodi : ti
Kelas : 1g
bobi berhasil ditambahkan ke antrian

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - aldi - ti - 1a
2. 124 - bobi - ti - 1g
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
Melayani mahasiswa: 123 - aldi - ti - 1a

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 124 - bobi - ti - 1g

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 5
Jumlah dalam antrian: 1

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 0
Terima kasih.
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 2\Algoritma &  Struktur Data\Praktik Algo&StrukturData>
```

2.2.3 Pertanyaan

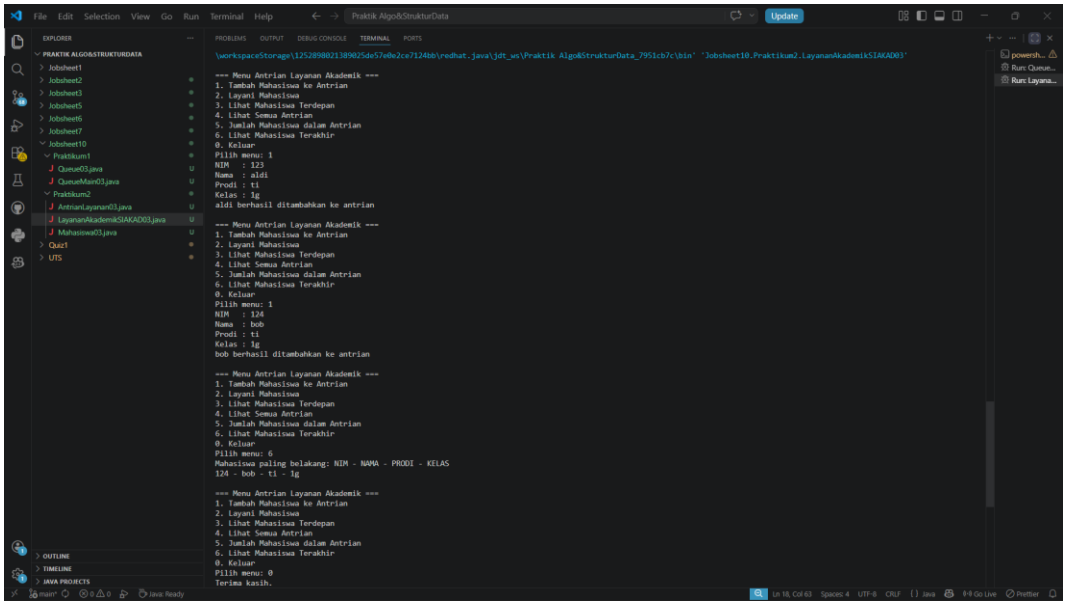
Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIAKAD sehingga method LihatAkhir dapat dipanggil!

Jawab

Tambah kode berikut di class AntrianLayanan03

```java
public void lihatAkhir() {
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            System.out.print("Mahasiswa paling belakang: ");
            System.out.println("NIM - NAMA - PRODI - KELAS");
            data[rear].tampilkanData();
        }
    }
```

Hasilnya:



2.3 Tugas

Buatlah program antrian untuk mengilustasikan antrian persetujuan Kartu Rencana Studi (KRS) Mahasiswa oleh Dosen Pembina Akademik (DPA). Ketika seorang mahasiswa akan mengantri, maka dia harus mendaftarkan datanya (data mahasiswa seperti pada praktikum 2). Gunakan class untuk antrian seperti pada Praktikum 1 dan 2, dengan method-method yang berfungsi :

- Cek antrian kosong, Cek antrian penuh, Mengosongkan antrian.
- Menambahkan antrian, Memanggil antrian untuk proses KRS – setiap 1x panggilan terdiri dari 2 mahasiswa (pada antrian no 1 dan 2)
- Menampilkan semua antrian, Menampilkan 2 antrian terdepan, Menampilkan antrian paling akhir.
- Cetak jumlah antrian, Cetak jumlah yang sudah melakukan proses KRS
- Jumlah antrian maximal 10, jumlah yang ditangani masing-masing DPA 30 mahasiswa, cetak jumlah mahasiswa yang belum melakukan proses KRS.

Gambarkan Diagram Class untuk antriannya. Implementasikan semua method menggunakan menu pilihan pada fungsi main.

Jawab

Mahasiswa03
nim: String nama: String prodi: String kelas: String
Mahasiswa03(nim: String, nama: String, prodi: String, kelas: String) tampilkanData(): void

AntrianKRS03
data: Mahasiswa03[] front: int rear: int size: int max: int sudahProses: int TOTAL_MAHASISWA_DPA: int = 30
AntrianKRS03(n: int) isEmpty(): boolean isFull(): boolean clear(): void tambahAntrian(mhs: Mahasiswa03): void prosesKRS(): void tampilkanSemua(): void tampilkanDuaTerdepan(): void tampilkanAkhir(): void cetakStatistik(): void

MainKRS03
main(args: String[]): void

Kode Program

- **Mahasiswa03**

```
package Jobsheet10.Tugas;

public class Mahasiswa03 {
    String nim, nama, prodi, kelas;

    public Mahasiswa03(String nim, String nama, String prodi, String kelas)
{
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
```

```
        System.out.println("NIM: " + nim + " | Nama: " + nama + " | Prodi: "
+ prodi + " | Kelas: " + kelas);
    }
}
```

- **AntrianKRS03**

```
package Jobsheet10.Tugas;

public class AntrianKRS03 {
    Mahasiswa03[] data;
    int front, rear, size, max;
    int sudahProses = 0;
    final int TOTAL_MAHASISWA_DPA = 30; // Kuota maksimal per DPA

    public AntrianKRS03(int n) {
        max = n;
        data = new Mahasiswa03[max];
        size = 0;
        front = 0;
        rear = -1;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == max;
    }

    public void clear() {
        front = 0;
        rear = -1;
        size = 0;
        System.out.println("Antrian berhasil dikosongkan.");
    }

    public void tambahAntrian(Mahasiswa03 mhs) {
        if (isFull()) {
            System.out.println("Antrian penuh! (Maksimal 10)");
        } else if (sudahProses + size >= TOTAL_MAHASISWA_DPA) {
            System.out.println("Mohon maaf, total kuota mahasiswa DPA (30
orang) sudah terpenuhi.");
```

```
        } else {
            rear = (rear + 1) % max;
            data[rear] = mhs;
            size++;
            System.out.println(mhs.nama + " berhasil masuk antrian.");
        }
    }

    public void prosesKRS() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            int jumlahDipanggil = (size >= 2) ? 2 : 1; // Panggil 2 jika tersedia,
jika tidak panggil 1
            System.out.println("--- Memproses " + jumlahDipanggil + "
Mahasiswa ---");
            for (int i = 0; i < jumlahDipanggil; i++) {
                Mahasiswa03 m = data[front];
                System.out.print("Memproses: ");
                m.tampilkanData();
                front = (front + 1) % max;
                size--;
                sudahProses++;
            }
        }
    }

    public void tampilkanSemua() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            for (int i = 0; i < size; i++) {
                int index = (front + i) % max;
                System.out.print((i + 1) + ". ");
                data[index].tampilkanData();
            }
        }
    }

    public void tampilkanDuaTerdepan() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
```

```
        } else {
            System.out.println("1. "); data[front].tampilkanData();
            if (size > 1) {
                System.out.println("2. "); data[(front + 1) % max].tampilkanData();
            }
        }
    }

    public void tampilkanAkhir() {
        if (isEmpty()) {
            System.out.println("Antrian kosong.");
        } else {
            data[rear].tampilkanData();
        }
    }

    public void cetakStatistik() {
        System.out.println("Jumlah antrian saat ini: " + size);
        System.out.println("Sudah melakukan KRS: " + sudahProses);
        System.out.println("Belum melakukan KRS (Sisa kuota DPA): " + (TOTAL_MAHASISWA_DPA - sudahProses));
    }
}
```

- **MainKRS03**

```
package Jobsheet10.Tugas;

import java.util.Scanner;

public class MainKRS03 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianKRS03 antrian = new AntrianKRS03(10);
        int pilih;

        do {
            System.out.println("\n--- MENU ANTRIAN KRS DPA ---");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Panggil/Proses KRS (2 Mahasiswa)");
            System.out.println("3. Tampilkan Semua Antrian");
            System.out.println("4. Tampilkan 2 Antrian Terdepan");
            System.out.println("5. Tampilkan Antrian Terakhir");
```

```
            System.out.println("6. Cetak Statistik & Jumlah");
            System.out.println("7. Kosongkan Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print("NIM: "); String nim = sc.nextLine();
                    System.out.print("Nama: "); String nama = sc.nextLine();
                    System.out.print("Prodi: "); String prodi = sc.nextLine();
                    System.out.print("Kelas: "); String kelas = sc.nextLine();
                    antrian.tambahAntrian(new  Mahasiswa03(nim,  nama,  prodi,
kelas));
                    break;
                case 2: antrian.prosesKRS(); break;
                case 3: antrian.tampilkanSemua(); break;
                case 4: antrian.tampilkanDuaTerdepan(); break;
                case 5: antrian.tampilkanAkhir(); break;
                case 6: antrian.cetakStatistik(); break;
                case 7: antrian.clear(); break;
            }
        } while (pilih != 0);
        sc.close();
    }
}
```